

Homework #1

Name(s): Raj Shah (ras637),

()

()

Problem 1. This question reviews asymptotic notation. You may assume the following inequalities for this question (and throughout the course): For any constant $c \geq 1$,

$$(\log n)^c = o(n), \quad n^c = o(n^{c+1}), \quad c^n = o((c+1)^n), \quad (n/2)^{(n/2)} = o(n!).$$

Question (a). Rank the functions in increasing asymptotic order and prove each adjacent $f_i = O(f_{i+1})$. (20 points)

Solution. Answer (a) Ranking (increasing order).

$$\log^3 n < \sqrt{n} < \frac{n}{\log n} < n^{1.5} < n^2 < (\log n)^{\log n} = n^{\log \log n} < n^{\log n} < e^{\sqrt{n}} < e^n < 3^n < 10^n < n! < n^n < 2^{n^2}.$$

Proofs for adjacent steps.

1. $\log^3 n = O(\sqrt{n})$. We compare by a limit:

$$\lim_{n \rightarrow \infty} \frac{\log^3 n}{\sqrt{n}} = \lim_{n \rightarrow \infty} \frac{(\log n)^3}{n^{1/2}} = 0.$$

We used that $(\log n)^c = o(n^\alpha)$ for any constants $c \geq 1$ and $\alpha > 0$. Since the ratio goes to 0, $\log^3 n = o(\sqrt{n})$, so $\log^3 n = O(\sqrt{n})$.

2. $\sqrt{n} = O\left(\frac{n}{\log n}\right)$. We compute the ratio:

$$\frac{\sqrt{n}}{n/\log n} = \frac{\log n}{\sqrt{n}}.$$

Then

$$\lim_{n \rightarrow \infty} \frac{\log n}{\sqrt{n}} = 0$$

because $\log n = o(n^{1/2})$. So $\sqrt{n} = o(n/\log n)$, hence $\sqrt{n} = O(n/\log n)$.

3. $\frac{n}{\log n} = O(n^{1.5})$.

$$\frac{(n/\log n)}{n^{1.5}} = \frac{1}{n^{1/2} \log n} \xrightarrow{n \rightarrow \infty} 0.$$

So $n/\log n = o(n^{1.5})$, hence $n/\log n = O(n^{1.5})$.

4. $n^{1.5} = O(n^2)$.

$$\frac{n^{1.5}}{n^2} = n^{-1/2} \xrightarrow{n \rightarrow \infty} 0.$$

So $n^{1.5} = o(n^2)$, hence $n^{1.5} = O(n^2)$.

5. $n^2 = O(n^{\log \log n})$. Since $\log \log n \rightarrow \infty$, choose N so that $\log \log n \geq 2$ for all $n \geq N$. Then for $n \geq N$,

$$n^{\log \log n} \geq n^2.$$

Equivalently,

$$\frac{n^2}{n^{\log \log n}} = n^{2 - \log \log n} \leq 1.$$

So $n^2 \leq 1 \cdot n^{\log \log n}$ for large n , hence $n^2 = O(n^{\log \log n})$.

6. $(\log n)^{\log n} = n^{\log \log n}$. We rewrite both sides:

$$(\log n)^{\log n} = e^{\ln((\log n)^{\log n})} = e^{(\log n) \ln(\log n)} = e^{(\log n)(\log \log n)}.$$

Also,

$$n^{\log \log n} = e^{\ln(n^{\log \log n})} = e^{(\log \log n) \ln n} = e^{(\log \log n)(\log n)}.$$

They match, so $(\log n)^{\log n} = n^{\log \log n}$ for all $n > 1$, and therefore they are $\Theta(\cdot)$ of each other.

7. $n^{\log \log n} = O(n^{\log n})$.

$$\frac{n^{\log \log n}}{n^{\log n}} = n^{\log \log n - \log n} = n^{-(\log n - \log \log n)} \rightarrow 0,$$

since $\log n - \log \log n \rightarrow \infty$. So $n^{\log \log n} = o(n^{\log n})$.

8. $n^{\log n} = O(e^{\sqrt{n}})$. Take logs:

$$\ln(n^{\log n}) = (\log n) \ln n = \Theta((\ln n)^2), \quad \ln(e^{\sqrt{n}}) = \sqrt{n}.$$

Since $(\ln n)^2 = o(\sqrt{n})$, we get $n^{\log n} = o(e^{\sqrt{n}})$.

9. $e^{\sqrt{n}} = O(e^n)$.

$$\frac{e^{\sqrt{n}}}{e^n} = e^{\sqrt{n} - n} = e^{-(n - \sqrt{n})} \rightarrow 0.$$

So $e^{\sqrt{n}} = o(e^n)$.

10. $e^n = O(3^n)$.

$$\frac{e^n}{3^n} = \left(\frac{e}{3}\right)^n \rightarrow 0 \quad (\text{since } e < 3).$$

So $e^n = o(3^n)$.

11. $3^n = O(10^n)$.

$$\frac{3^n}{10^n} = \left(\frac{3}{10}\right)^n \rightarrow 0.$$

So $3^n = o(10^n)$.

12. $10^n = O(n!)$. For $n \geq 200$, $\frac{n}{2} \geq 100$ and so

$$\left(\frac{n}{2}\right)^{n/2} \geq 100^{n/2} = 10^n.$$

Given $\left(\frac{n}{2}\right)^{n/2} = o(n!)$, we have $10^n = o(n!)$, hence $10^n = O(n!)$.

13. $n! = O(n^n)$.

$$n! = 1 \cdot 2 \cdots n \leq n \cdot n \cdots n = n^n.$$

So $n! = O(n^n)$.

14. $n^n = O(2^{n^2})$. Take logs:

$$\ln(n^n) = n \ln n, \quad \ln(2^{n^2}) = n^2 \ln 2.$$

Since $n \ln n = o(n^2)$, we get $\frac{n \ln n}{n^2} \rightarrow 0$, so $n^n = o(2^{n^2})$.

Question (b). For each $f(n)$, decide which statements are true or false and prove your choices. (10 points)

Solution. Let S1 be $f(n) = \Theta(f(n-1))$, S2 be $f(n) = \Theta(f(n/2))$, and S3 be $f(n) = \Theta(f(\sqrt{n}))$.

1. $f(n) = 1$. S1 True, S2 True, S3 True.

We have $f(n) = f(n-1) = f(n/2) = f(\sqrt{n}) = 1$. So every ratio equals 1. That means each pair is within constant factors. Therefore all three statements are true.

2. $f(n) = \log^4 n$. S1 True, S2 True, S3 True.

For S1,

$$\lim_{n \rightarrow \infty} \frac{\log^4 n}{\log^4(n-1)} = \lim_{n \rightarrow \infty} \left(\frac{\log n}{\log(n-1)} \right)^4 = 1,$$

so S1 is true. For S2,

$$\frac{\log^4 n}{\log^4(n/2)} = \left(\frac{\log n}{\log n - \log 2} \right)^4 \rightarrow 1,$$

so S2 is true. For S3,

$$\frac{\log^4 n}{\log^4(\sqrt{n})} = \left(\frac{\log n}{\frac{1}{2} \log n} \right)^4 = 16,$$

a constant, so S3 is true.

3. $f(n) = n^{\sqrt{\log n}}$. S1 True, S2 False, S3 False.

For S1, we compare logs of the ratio:

$$\ln \left(\frac{f(n)}{f(n-1)} \right) = \sqrt{\log n} \ln n - \sqrt{\log(n-1)} \ln(n-1) \rightarrow 0,$$

so the ratio $\rightarrow 1$ and S1 is true. For S2,

$$\ln \left(\frac{f(n)}{f(n/2)} \right) = \sqrt{\log n} \ln n - \sqrt{\log(n/2)} \ln(n/2) \rightarrow \infty,$$

so $\frac{f(n)}{f(n/2)} \rightarrow \infty$ and S2 is false. For S3,

$$\ln \left(\frac{f(n)}{f(\sqrt{n})} \right) = \sqrt{\log n} \ln n - \sqrt{\frac{1}{2} \log n} \cdot \frac{1}{2} \ln n = \left(1 - \frac{1}{2\sqrt{2}} \right) \sqrt{\log n} \ln n \rightarrow \infty,$$

so $\frac{f(n)}{f(\sqrt{n})} \rightarrow \infty$ and S3 is false.

4. $f(n) = 5^{n^3}$. S1 False, S2 False, S3 False.

For S1,

$$\frac{f(n)}{f(n-1)} = \frac{5^{n^3}}{5^{(n-1)^3}} = 5^{3n^2-3n+1} \rightarrow \infty,$$

so S1 is false. For S2,

$$\frac{f(n)}{f(n/2)} = 5^{n^3-(n/2)^3} = 5^{(7/8)n^3} \rightarrow \infty,$$

so S2 is false. For S3,

$$\frac{f(n)}{f(\sqrt{n})} = 5^{n^3-(\sqrt{n})^3} = 5^{n^3-n^{3/2}} \rightarrow \infty,$$

so S3 is false.

Question (c). For each pair, state whether $f(n) = O(g(n))$, $f(n) = \Omega(g(n))$, or $f(n) = \Theta(g(n))$. No proofs required. **(15 points)**

Solution. We write each pair and the relation.

- a) $f(n) = \sum_{k=1}^n \log k$ and $g(n) = \log(n!)$: Since $\log(n!) = \log(\prod_{k=1}^n k) = \sum_{k=1}^n \log k$, we have $f = \Theta(g)$.
 - b) $f(n) = n^{\log_3 n}$ and $g(n) = n^{\sqrt{\log n}}$: Because $\log_3 n = \Theta(\log n)$ and $\log n \gg \sqrt{\log n}$, we get $f = \Omega(g)$.
 - c) $f(n) = \sum_{k=1}^n k^2$ and $g(n) = n^3$: We know $\sum_{k=1}^n k^2 = \Theta(n^3)$, so $f = \Theta(g)$.
 - d) $f(n) = 2^{\sqrt{n \log n}}$ and $g(n) = 3^{\sqrt{n}}$: Since $\sqrt{n \log n} = \sqrt{n} \sqrt{\log n}$ and $\sqrt{\log n} \rightarrow \infty$, f grows faster, so $f = \Omega(g)$.
 - e) $f(n) = n^{\log n}$ and $g(n) = n^{\log \log n}$: Because $\log n \gg \log \log n$, we get $f = \Omega(g)$.
 - f) $f(n) = 10^n$ and $g(n) = n^{n/2}$: We can write $g(n) = (n^{1/2})^n$. For large n , $n^{1/2} > 10$, so $10^n = O((n^{1/2})^n)$ and $f = O(g)$.
 - g) $f(n) = n!$ and $g(n) = (n/e)^n$: By Stirling, $n! = \Theta(\sqrt{n} (n/e)^n)$, so $f = \Omega(g)$.
 - h) $f(n) = \sum_{k=1}^n k \log k$ and $g(n) = \frac{n^2}{\log n}$: We have $\sum_{k=1}^n k \log k = \Theta(n^2 \log n)$, so $f = \Omega(g)$.
 - i) $f(n) = (\log n)^{\log n}$ and $g(n) = 2^{\log n}$: We rewrite $f(n) = (\log n)^{\log n} = n^{\log \log n}$ and $g(n) = 2^{\log n} = n^{\log 2}$. Since $\log \log n \rightarrow \infty$, f grows faster, so $f = \Omega(g)$.
 - j) $f(n) = e^{\sqrt{n}}$ and $g(n) = 2^n$: Because $\sqrt{n} = o(n)$, $e^{\sqrt{n}}$ is much smaller than 2^n , so $f = O(g)$.
-

Problem 2. Your goal in this problem is to analyze the runtime of each recursive algorithm by writing a recurrence and solving it using a recursion tree. **(25 points)**

Question (A). Algorithm A splits into 5 subproblems of size $n/5$ and combines in $O(n)$.

Solution. Answer (A). We write the recurrence

$$T(n) = 5T(n/5) + cn.$$

At level i , there are 5^i subproblems of size $n/5^i$, so the total combine work on that level is

$$5^i \cdot c \left(\frac{n}{5^i} \right) = cn.$$

The depth is when $n/5^h = 1$, so $h = \log_5 n$. Therefore the total work is

$$\sum_{i=0}^{\log_5 n} cn = cn(\log_5 n + 1) = \Theta(n \log n),$$

so $T(n) = \Theta(n \log n)$.

Question (B). Algorithm B splits into 5 subproblems of size $n/2$ and combines in $O(n^2)$.

Solution. Answer (B). We write the recurrence

$$T(n) = 5T(n/2) + cn^2.$$

At level i , there are 5^i subproblems of size $n/2^i$, so the total combine work is

$$5^i \cdot c \left(\frac{n}{2^i} \right)^2 = cn^2 \left(\frac{5}{4} \right)^i.$$

Since $\frac{5}{4} > 1$, the level costs grow as i increases, so the last level dominates. The depth is $h = \log_2 n$, so the last level cost is

$$cn^2 \left(\frac{5}{4} \right)^{\log_2 n} = cn^2 \cdot n^{\log_2(5/4)} = cn^{2+\log_2(5/4)} = cn^{\log_2 5}.$$

Therefore $T(n) = \Theta(n^{\log_2 5})$.

Question (C). Algorithm C splits into 5 subproblems of size $n/2$ and combines in $O(n^{0.8})$.

Solution. Answer (C). We write the recurrence

$$T(n) = 5T(n/2) + cn^{0.8}.$$

At level i , there are 5^i subproblems of size $n/2^i$, so the total combine work is

$$5^i \cdot c \left(\frac{n}{2^i} \right)^{0.8} = cn^{0.8} \left(\frac{5}{2^{0.8}} \right)^i.$$

Since $\frac{5}{2^{0.8}} > 1$, the level costs increase, so the bottom level dominates again. With depth $h = \log_2 n$, the bottom level cost is

$$cn^{0.8} \left(\frac{5}{2^{0.8}} \right)^{\log_2 n} = cn^{0.8} \cdot n^{\log_2(5/2^{0.8})} = cn^{0.8} \cdot n^{\log_2 5 - 0.8} = cn^{\log_2 5}.$$

So $T(n) = \Theta(n^{\log_2 5})$.

Question (D). Algorithm D makes subproblems of sizes $n/4$ and $n/5$ and combines in $O(n)$.

Solution. Answer (D). We write the recurrence

$$T(n) = T(n/4) + T(n/5) + cn.$$

At the next level, the total input size across children is

$$\frac{n}{4} + \frac{n}{5} = \frac{9}{20}n,$$

and since $\frac{9}{20} < 1$, the total size shrinks by a constant factor each level. So the total combine work over levels is

$$cn + c \left(\frac{9}{20}n \right) + c \left(\frac{9}{20} \right)^2 n + \cdots = cn \sum_{i=0}^{\infty} \left(\frac{9}{20} \right)^i = cn \cdot \frac{1}{1 - \frac{9}{20}} = O(n).$$

We also have at least $\Omega(n)$ work from the top combine step, so overall $T(n) = \Theta(n)$.

Question (E). Algorithm E makes 3 subproblems of size $n - 1$ and combines in $O(1)$.

Solution. Answer (E). We write the recurrence

$$T(n) = 3T(n - 1) + c.$$

Expanding:

$$T(n) = 3(3T(n - 2) + c) + c = 3^2T(n - 2) + 3c + c.$$

Continuing for k steps gives

$$T(n) = 3^kT(n - k) + c(1 + 3 + 3^2 + \cdots + 3^{k-1}).$$

Taking $k = n$ reaches the base case and the geometric sum is $\Theta(3^n)$, so $T(n) = \Theta(3^n)$.

Problem 3. For each algorithm, state the running time in Θ -notation.

(15 points)

Question (a). For $i = 1$ to $n/3$, set $j = 10$, while $j \leq i/20$ do $O(1)$ and $j \leftarrow j + 10$.

Solution. Answer (a). For fixed i , j takes values $10, 20, 30, \dots$ up to about $i/20$, so the number of iterations is

$$\Theta\left(\frac{i/20}{10}\right) = \Theta(i).$$

Therefore total work is

$$\sum_{i=1}^{n/3} \Theta(i) = \Theta\left(\left(\frac{n}{3}\right)^2\right) = \Theta(n^2).$$

So the runtime is $\Theta(n^2)$.

Question (b). While $n \geq 1$, set $n \leftarrow n/3$ and do $O(1)$ work.

Solution. Answer (b). After t iterations, n becomes $n/3^t$. We stop when $n/3^t < 1$, so $3^t > n$ and

$$t > \log_3 n.$$

Thus the number of iterations is $\Theta(\log n)$, so runtime is $\Theta(\log n)$.

Question (c). While $n \geq 2$, set $n \leftarrow \sqrt{n}$ and do $O(1)$ work.

Solution. Answer (c). After t iterations, n becomes $n^{1/2^t}$. We stop when $n^{1/2^t} < 2$. Taking $\log_2$:

$$\frac{1}{2^t} \log_2 n < 1 \quad \Rightarrow \quad 2^t > \log_2 n \quad \Rightarrow \quad t > \log_2 \log_2 n.$$

So the runtime is $\Theta(\log \log n)$.

Question (d). For $i = 1$ to n , for $j = 1$ to i^2 do $O(1)$.

Solution. Answer (d). Total work is

$$\sum_{i=1}^n i^2 = \Theta(n^3),$$

so runtime is $\Theta(n^3)$.

Question (e). For $i = 1$ to n , for $j = 1$ to $\log i$, for $k = 1$ to n do $O(1)$.

Solution. Answer (e). Work for each i is $\Theta(\log i) \cdot \Theta(n) = \Theta(n \log i)$, so total is

$$\sum_{i=1}^n \Theta(n \log i) = \Theta\left(n \sum_{i=1}^n \log i\right) = \Theta(n \cdot n \log n) = \Theta(n^2 \log n).$$

So runtime is $\Theta(n^2 \log n)$.

Problem 4. Given

$$T(n) = 2T(n/2) + T(n/2 + 1) + O(n),$$

prove the carry issue does not affect the asymptotic complexity and we still have $O(n^{\log_2 3})$. **(20 points)**

Question. Prove that the recurrence still gives $O(n^{\log_2 3})$.

Solution. Answer. Let $a = \log_2 3$, so $2^a = 3$. We assume by induction that for all smaller inputs $m < n$ we have $T(m) \leq Cm^a$ for a constant C . Using the recurrence, we get

$$T(n) \leq 2C \left(\frac{n}{2}\right)^a + C \left(\frac{n}{2} + 1\right)^a + cn.$$

We rewrite

$$\left(\frac{n}{2} + 1\right)^a = \left(\frac{n}{2}\right)^a \left(1 + \frac{2}{n}\right)^a \leq \left(\frac{n}{2}\right)^a \left(1 + \frac{K}{n}\right)$$

for some constant K and all large n . Then

$$T(n) \leq C \left(\frac{n}{2}\right)^a \left(2 + 1 + \frac{K}{n}\right) + cn = 3C \left(\frac{n}{2}\right)^a + O(n^{a-1}) + cn.$$

Since $3 \left(\frac{n}{2}\right)^a = 3 \cdot \frac{n^a}{2^a} = n^a$ and $n^{a-1} = o(n)$, we can choose C large enough so that the $O(n)$ terms are bounded by Cn^a for all sufficiently large n . Therefore $T(n) \leq Cn^a$ for large n , which proves

$$T(n) = O(n^{\log_2 3}).$$

Problem 5. Toom-3 splits two n -digit numbers into three parts.

1. Write the recurrence for the simple divide and conquer method (9 multiplications) and solve it.
2. Toom-3 uses 5 multiplications, find its time complexity.

(20 points)

Question (1). Simple divide and conquer (9 multiplications).

Solution. Answer (1). We do 9 multiplications of size $n/3$ plus linear overhead, so

$$T(n) = 9T(n/3) + cn.$$

Here $a = 9$ and $b = 3$, so $n^{\log_b a} = n^{\log_3 9} = n^2$. Since $cn = O(n^{2-\epsilon})$ with $\epsilon = 1$, we get

$$T(n) = \Theta(n^2).$$

Question (2). Toom-3 (5 multiplications).

Solution. Answer (2). Toom-3 uses 5 multiplications of size $n/3$ plus linear overhead, so

$$T(n) = 5T(n/3) + cn.$$

Now $n^{\log_b a} = n^{\log_3 5}$. Since $cn = O(n^{\log_3 5 - \epsilon})$ for some $\epsilon > 0$, the recursion dominates and

$$T(n) = \Theta(n^{\log_3 5}) \approx \Theta(n^{1.465}).$$

Challenge Yourself. We have an n -story building and a critical level L where vases start breaking. Design asymptotically optimal strategies minimizing the number of vase throws.

(a) One vase. (+2 points)

(b) Four vases. (+4 points)

(c) Unlimited vases. (+4 points)

Solution. Answer (a). With one vase, once it breaks we cannot test lower floors, so we must go in order from the bottom. We throw from floor $1, 2, 3, \dots$ until the first break. In the worst case it breaks at floor n , so we use $\Theta(n)$ throws.

Answer (b). With k vases and t throws, the maximum floors we can handle satisfies

$$F(t, k) = F(t - 1, k - 1) + F(t - 1, k) + 1,$$

which solves to

$$F(t, k) = \sum_{i=1}^k \binom{t}{i}.$$

For $k = 4$, we need the smallest t such that $\sum_{i=1}^4 \binom{t}{i} \geq n$. The dominant term is $\binom{t}{4} = \Theta(t^4)$, so we need $t = \Theta(n^{1/4})$. We can implement a decreasing-step strategy where the first drop is chosen so that if it breaks we have $t - 1$ drops and 3 vases left for the remaining lower floors, and if it does not break we move upward and repeat with one fewer allowed drop. This gives worst-case $\Theta(n^{1/4})$ throws.

Answer (c). With unlimited vases, we can binary search on floors. Each throw halves the remaining interval. So the number of throws is $\Theta(\log n)$.
